

# SpeakX-Pro: Complete Project Documentation

## 1. Executive Summary

**SpeakX-Pro** (formerly Confidence Coach) is an intelligent, privacy-first AI platform designed to help individuals dramatically improve their public speaking skills, communication fluency, and self-confidence through the automated analysis of video recordings.

The platform has been completely rebuilt from a legacy monolithic architecture into a modern, highly scalable full-stack application utilizing **FastAPI**, **PostgreSQL** with vector embeddings, and a sleek **React** frontend.

## 2. Technical Stack

- **Frontend:** React, Vite, TypeScript, Tailwind CSS, shadcn/ui.
- **Backend API:** Python, FastAPI, SQLAlchemy, Alembic (Migrations), JWT Authentication.
- **Machine Learning:**
  - **Transcription & Audio:** OpenAI Whisper, Librosa
  - **Video & Body Language:** Google MediaPipe (Face Mesh, Pose, Holistic)
  - **NLP:** NLTK, Textstat, LanguageTool
  - **Embeddings & LLM:** sentence-transformers (all-MiniLM-L6-v2), Ollama (Local RAG)
- **Database:** PostgreSQL with pgvector extension.

## 3. Database Architecture

The system utilizes a relational database (PostgreSQL) combined with a vector extension (pgvector) to store both structured user data and unstructured machine-learning embeddings.

### Entity-Relationship Breakdown

1. **users Table**
  - Core table for Authentication and Role-Based Access Control (RBAC).
  - Columns: id, username, email, password\_hash, role (student, admin, etc.), is\_active, created\_at.
2. **sessions Table**
  - Stores the complete extracted metric data from a single video upload.
  - Columns: id, user\_id (Foreign Key), session\_date, overall\_score, transcript, wpm, eye\_contact\_pct, and 15+ other detailed audio/video metrics
3. **session\_embeddings Table**
  - Stores the RAG (Retrieval-Augmented Generation) Chatbot context.
  - Columns: id, session\_id, user\_id, content, embedding (Vector 384 dims).
  - Uses pgvector to enable blazing-fast Cosine Similarity searches across past AI coaching feedback natively in SQL.
4. **chat\_history Table**
  - Stores the messages sent back and forth between the user and the AI Coach to maintain conversational memory.
  - Columns: id, user\_id, role, message, created\_at.

## 4. Frontend Features & Implementation

The React frontend handles all user interactions and visually presents the machine learning outcomes.

- **Authentication System:** Secure JWT-based login (/login) integrating a React AuthContext to persist sessions securely via localStorage.
- **Student Dashboard (/dashboard):** A premium, glassmorphic layout displaying Key Performance Indicators (KPIs). It fetches historical sessions to chart the user's progress and Overall Score trend over time using recharts.
- **Video Upload (/analyze):** A Drag-and-Drop file uploader that securely posts multipart/form-data to the FastAPI backend. It features real-time loading states and dynamically renders a Success Card highlighting the speaker's Words-Per-Minute, Eye Contact Percentage, and AI-generated Feedback upon completion.
- **AI Coach (/coach):** A ChatGPT-style interface enabling users to chat directly with their historical video data, powered by the backend RAG implementation.

## 5. Machine Learning & Backend Workflow

When a user uploads a video via the frontend, the following synchronous pipeline is triggered on the FastAPI backend:

1. **Extraction:** The video file is saved temporarily. Librosa extracts the audio track.
2. **Vision Analysis:** MediaPipe scans the video frames to calculate posture alignment, eye contact percentage, and dominant gestures.
3. **Audio Analysis:** Whisper transcribes the audio, while custom algorithms calculate speaking pace (WPM), pause durations, and filler word frequency.
4. **Linguistic Analysis:** NLTK and Textstat analyze the transcription for readability, vocabulary richness, and coherence.
5. **AI Synthesis:** The extracted metrics are sent to a local Ollama LLM to generate actionable, personalized coaching feedback.
6. **Vectorization:** The LLM feedback and transcript are embedded into a 384-dimensional vector using sentence-transformers and saved to pgvector in the database.
7. **Response:** The final consolidated score and metrics are returned to the React frontend.